*The EAOS methodology defines a structured lifecycle for Designing, deploying, and governing enterprise-grade AI systems*

## Project: Standard Framework Project

# 3.4 Memory Spec

### Phase: 3. Capability

### [산출물 ID: 3.4 Memory Spec.docx]

**WBS Task**

3.4.1 Memory 유형별 저장 구조 및 스키마 정의
3.4.2 Retrieval Rule (조회 우선순위•필터•만료 정책)
3.4.3 개인정보 처리 방침 및 Memory 생명주기 관리

| | |
|---|---|
| 문서 번호 | 3.4 Memory Spec.docx |
| 버전 | v1.0 |
| 작성일 | 2026 |
| 작성자 | KY Song |
| 분류 | CONFIDENTIAL |
| 적용 대상 | Standard Framework 프로젝트 전체 |

# Table of Contents

1. 개요 ..... 3
1.1 Memory 4계층 구조 ..... 3
1.2 Memory 유형별 특성 비교 ..... 3
2. Layer 1 — Context Window Memory ..... 4
2.1 정의 ..... 4
2.2 관리 규칙 ..... 4
2.3 민감정보 필터링 ..... 4
3. Layer 2 — Session Memory (Redis) ..... 5
3.1 저장 구조 ..... 5
3.2 Agent 실행 상태 스키마 ..... 5
3.3 Session Memory 갱신 규칙 ..... 5
4. Layer 3 — Episodic Memory (PostgreSQL) ..... 6
4.1 trajectory_logs 테이블 스키마 ..... 6
4.2 Episodic Memory 조회 규칙 ..... 7
4.3 데이터 보존 정책 ..... 7
5. Layer 4 — Semantic Memory (pgvector) ..... 8
5.1 저장 구조 ..... 8
5.2 Memory 유형별 정의 ..... 8
5.3 Semantic Memory Retrieval Rule ..... 9
5.4 Semantic Memory 등록 규칙 ..... 10
6. Memory 생명주기 관리 ..... 11
6.1 전체 생명주기 흐름 ..... 11
6.2 개인정보 처리 방침 ..... 11
6.3 Memory 무결성 보장 ..... 11
7. Agent별 Memory 사용 기준 ..... 12
8. 산출물 체크리스트 ..... 13
10. 승인 (Approval) ..... 14

# 1. 개요

Memory는 AI Agent가 과거 실행 이력, 사용자 컨텍스트, 도메인 지식을 유지하고 재활용하는 구조다. 적절한 Memory 설계 없이는 Agent가 대화 맥락을 잃거나, 동일 실수를 반복하거나, 불필요한 LLM 호출이 증가한다.

> 💡 Memory는 Agent의 "작업 기억(Working Memory)"과 "장기 기억(Long-term Memory)"을 모두 포함한다. 공공기관 적용 시스템 특성상 모든 Memory는 개인정보보호법 및 데이터 보존 정책의 적용을 받는다.

## 1.1 Memory 4 계층 구조

| | | |
|---|---|---|
| Layer 1. | Context Window Memory | 현재 대화 세션 내 임시 메모리 (LLM 토큰 컨텍스트) |
| Layer 2. | Session Memory | 세션 단위 작업 상태 (Redis, TTL: 24h) |
| Layer 3. | Episodic Memory | Agent 실행 에피소드 로그 (PostgreSQL, trajectory_logs) |
| Layer 4. | Semantic Memory | 도메인 지식 · 패턴 · 임베딩 (PostgreSQL + pgvector) |

## 1.2 Memory 유형별 특성 비교

| 구분 | Layer 1 | Layer 2 | Layer 3 | Layer 4 |
|---|---|---|---|---|
| 저장소 | LLM 컨텍스트 | Redis | PostgreSQL | PostgreSQL + pgvector |
| 범위 | 단일 대화 | 단일 세션 | 영구 | 영구 |
| TTL | 세션 종료 시 소멸 | 24h | 보존 정책 기준 | 버전 관리 |
| 조회 방식 | 자동 (LLM) | 키 조회 | SQL 쿼리 | 벡터 유사도 |
| 개인정보 포함 | 가능 | 가능 | 가능 | 익명화 |
| 승인 필요 | 불필요 | 불필요 | 읽기: 불필요 / 삭제: P1 | 수정: P1 |

# 2. Layer 1 — Context Window Memory

## 2.1 정의

LLM API 호출 시 messages 배열에 포함되는 대화 이력이다. Agent 가 현재 작업의 맥락을 유지하는 데 사용한다.

## 2.2 관리 규칙

```
class ContextWindowManager:
    MAX_TOKENS = 100_000          # 최대 컨텍스트 크기
    RESERVE_TOKENS = 8_192        # 출력용 예약 토큰
    WORKING_TOKENS = 91_808       # 실제 사용 가능 토큰

    def build_messages(
        self,
        system_prompt: str,
        history: list[dict],
        new_message: str,
    ) -> list[dict]:
        """토큰 초과 시 오래된 메시지부터 제거 (sliding window)"""
        messages = history.copy()
        messages.append({"role": "user", "content": new_message})

        while self._count_tokens(messages) > self.WORKING_TOKENS:
            # 가장 오래된 user-assistant 쌍 제거 (system 메시지 유지)
            if len(messages) > 2:
                messages.pop(1)
                messages.pop(1)
            else:
                break

        return messages
```

## 2.3 민감정보 필터링

```
SENSITIVE_PATTERNS = [
    r"\b\d{13}\b",                          # 주민등록번호 (13 자리)
    r"\b\d{4}-\d{4}-\d{4}-\d{4}\b",   # 신용카드 번호
    r"(?i)(password|secret|api_key)\s*[:=]\s*\S+",   # 비밀번호/키
]

def sanitize_context(content: str) -> str:
    """컨텍스트 저장 전 민감정보 마스킹"""
    for pattern in SENSITIVE_PATTERNS:
        content = re.sub(pattern, "[REDACTED]", content)
    return content
```

# 3. Layer 2 — Session Memory (Redis)

## 3.1 저장 구조

| Redis 키 | 저장 데이터 | TTL | 접근 Agent |
|---|---|---|---|
| agent_run:{trace_id} | 현재 실행 상태 스냅샷 | 1h | Orch, 해당 Agent |
| agent_context:{agent_id}:{session_id} | Agent 세션 컨텍스트 | 24h | 해당 Agent |
| approval_state:{approval_id} | 승인 요청 대기 상태 | 48h | Orch |
| workflow_state:{workflow_id} | Workflow 진행 상태 | 24h | Orch |
| tool_call_cache:{hash} | 동일 Tool 호출 캐시 | 5m | 해당 Agent |

## 3.2 Agent 실행 상태 스키마

```
class AgentRunState(BaseModel):
    """agent_run:{trace_id} 에 저장되는 구조"""
    trace_id: UUID
    agent_id: str
    status: Literal["initializing", "running", "waiting_approval",
                    "escalated", "completed", "failed"]
    current_step: str
    steps_completed: list[str]
    context_summary: str            # LLM 요약 (토큰 절약)
    pending_approval_id: UUID | None
    error_count: int = 0
    started_at: datetime
    last_updated_at: datetime

    class Config:
        json_encoders = {UUID: str, datetime: lambda v: v.isoformat()}
```

## 3.3 Session Memory 갱신 규칙

```
Agent 실행 단계마다:
  1. Redis 에서 현재 상태 조회 (GET agent_run:{trace_id})
  2. 작업 수행
  3. 상태 갱신 후 저장 (SET agent_run:{trace_id} + EXPIRE 3600)
  4. audit_log 에 상태 변경 기록

세션 종료 시:
  1. 최종 상태를 PostgreSQL trajectory_logs 에 영구 저장
  2. Redis 키 삭제 (DEL agent_run:{trace_id})
```

# 4. Layer 3 — Episodic Memory (PostgreSQL)

## 4.1 trajectory_logs 테이블 스키마

```
CREATE TABLE trajectory_logs (
    id                  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    trace_id            UUID NOT NULL UNIQUE,
    agent_id            VARCHAR(50) NOT NULL,
    session_id          UUID,
    workflow_id         UUID,

    -- 실행 입력
    input_summary       TEXT,               -- 민감정보 제거 후 요약
    input_tokens        INTEGER,

    -- 실행 경로
    steps               JSONB NOT NULL DEFAULT '[]',
    -- [{"step_name": str, "status": str, "duration_ms": int, "tool_called": str|null}]

    -- 실행 결과
    output_summary      TEXT,
    output_tokens       INTEGER,
    status              VARCHAR(20) NOT NULL,   -- "success|failure|partial|escalated"
    failure_reason      TEXT,
    failure_tags        TEXT[],                     -- ["role_conflict", "tool_timeout", ...]

    -- HITL 관련
    approval_required   BOOLEAN DEFAULT FALSE,
    approval_id         UUID,
    approval_status     VARCHAR(20),            -- "pending|approved|rejected|timeout"

    -- 메타데이터
    kpi_snapshot        JSONB DEFAULT '{}',
    metadata            JSONB DEFAULT '{}',

    -- 타임스탬프
    started_at          TIMESTAMPTZ NOT NULL,
    completed_at        TIMESTAMPTZ,
    duration_ms         INTEGER,

    created_at          TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_trajectory_agent_id ON trajectory_logs(agent_id);
CREATE INDEX idx_trajectory_status ON trajectory_logs(status);
CREATE INDEX idx_trajectory_started_at ON trajectory_logs(started_at DESC);
CREATE INDEX idx_trajectory_failure_tags ON trajectory_logs USING GIN(failure_tags);
```

## 4.2 Episodic Memory 조회 규칙

```
class EpisodicMemoryRetriever:

    async def get_recent_failures(
        self,
        agent_id: str,
        limit: int = 10,
        days: int = 7,
    ) -> list[TrajectoryLog]:
        """최근 실패 에피소드 조회 — Evol-A 분석에 사용"""
        return await self.repo.query(
            agent_id=agent_id,
            status="failure",
            from_date=datetime.now() - timedelta(days=days),
            limit=limit,
            order_by="started_at DESC",
        )

    async def get_similar_episodes(
        self,
        input_summary: str,
        agent_id: str,
        limit: int = 5,
    ) -> list[TrajectoryLog]:
        """유사한 과거 에피소드 조회 — Layer 4 벡터 검색 활용"""
        embedding = await self.embedding_service.encode(input_summary)
        return await self.repo.vector_search(
            embedding=embedding,
            agent_id=agent_id,
            limit=limit,
            min_similarity=0.85,
        )
```

## 4.3 데이터 보존 정책

| 데이터 유형 | 보존 기간 | 만료 처리 |
|---|---|---|
| 성공 에피소드 (일반) | 1 년 | 자동 삭제 |
| 실패 에피소드 | 2 년 | 자동 삭제 (Evol-A 분석 완료 후) |
| CRITICAL 에피소드 | 5 년 | 수동 검토 후 삭제 (P1 필수) |
| 감사 목적 에피소드 | 법정 보존 기간 (7 년) | 삭제 불가 |
| 승인 요청 이력 | 3 년 | 자동 아카이빙 |

# 5. Layer 4 — Semantic Memory (pgvector)

## 5.1 저장 구조

Semantic Memory는 Agent가 도메인 지식, 패턴, 개선 이력을 장기적으로 학습하고 재사용하기 위한 벡터 저장소다.

```
-- pgvector 확장 필요
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE semantic_memories (
    id                  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    agent_id            VARCHAR(50),            -- NULL이면 전체 공유 지식
    memory_type         VARCHAR(50) NOT NULL, -- "knowledge|pattern|improvement|constraint"
    content             TEXT NOT NULL,          -- 저장할 지식 내용 (익명화)
    embedding           vector(1536),           -- OpenAI text-embedding-3-small 기준
    source_trace_id UUID,                       -- 학습 출처 trajectory_log
    tags                TEXT[] DEFAULT '{}',
    confidence          DECIMAL(3,2),           -- 0.00 ~ 1.00
    use_count           INTEGER DEFAULT 0,
    is_active           BOOLEAN DEFAULT TRUE,
    version             INTEGER DEFAULT 1,
    created_at          TIMESTAMPTZ DEFAULT NOW(),
    updated_at          TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_semantic_embedding ON semantic_memories
    USING ivfflat (embedding vector_cosine_ops) WITH (lists = 100);
CREATE INDEX idx_semantic_agent_type ON semantic_memories(agent_id, memory_type);
CREATE INDEX idx_semantic_tags ON semantic_memories USING GIN(tags);
```

## 5.2 Memory 유형별 정의

| memory_type | 설명 | 학습 주체 | 적용 대상 |
|---|---|---|---|
| knowledge | 도메인 규칙·정책·제약사항 | 사전 등록 (Human) | 전체 Agent |
| pattern | 반복되는 성공 실행 패턴 | Evol-A (자동) | 해당 Agent |
| improvement | 개선 제안 및 적용 이력 | Evol-A (승인 후) | 해당 Agent |
| constraint | 금지 패턴·실패 원인 | Evol-A + Test-A | 전체 Agent |

## 5.3 Semantic Memory Retrieval Rule

```
class SemanticMemoryRetriever:

    RETRIEVAL_CONFIG = {
        "max_results": 5,
        "min_similarity": 0.80,       # 80% 이상 유사도만 반환
        "max_tokens_budget": 2000,   # 컨텍스트 주입 시 최대 토큰
    }

    async def retrieve(
        self,
        query: str,
        agent_id: str,
        memory_types: list[str] | None = None,
    ) -> list[SemanticMemory]:
        """
        조회 우선순위:
        1. agent_id 일치 + 높은 유사도
        2. agent_id = NULL (공유 지식) + 높은 유사도
        3. confidence 높은 순
        4. use_count 높은 순 (자주 쓰인 지식 우선)
        """
        embedding = await self.embed(query)

        results = await self.db.execute("""
            SELECT *, 1 - (embedding <=> :embedding) AS similarity
            FROM semantic_memories
            WHERE is_active = TRUE
              AND (agent_id = :agent_id OR agent_id IS NULL)
              AND (:memory_types IS NULL OR memory_type = ANY(:memory_types))
              AND 1 - (embedding <=> :embedding) >= :min_similarity
            ORDER BY
                CASE WHEN agent_id = :agent_id THEN 0 ELSE 1 END,
                similarity DESC,
                confidence DESC,
                use_count DESC
            LIMIT :max_results
        """, {
            "embedding": embedding,
            "agent_id": agent_id,
            "memory_types": memory_types,
            "min_similarity": self.RETRIEVAL_CONFIG["min_similarity"],
            "max_results": self.RETRIEVAL_CONFIG["max_results"],
        })

        # 조회 횟수 갱신
        await self.db.execute(
            "UPDATE semantic_memories SET use_count = use_count + 1 WHERE id = ANY(:ids)",
            {"ids": [r.id for r in results]}
        )

        return results
```

## 5.4 Semantic Memory 등록 규칙

```
class SemanticMemoryWriter:

    async def register_knowledge(
        self,
        content: str,
        memory_type: str,
        agent_id: str | None,
        source_trace_id: UUID | None,
        approval_id: UUID,              # P1 승인 필수
        tags: list[str] = [],
        confidence: float = 0.9,
    ) -> SemanticMemory:
        """
        Semantic Memory 신규 등록 — Evol-A 또는 Human만 가능
        반드시 P1 ApprovalGate(G-09) 통과 후 호출
        """
        # 중복 확인 (유사도 95% 이상이면 기존 업데이트)
        existing = await self.retrieve(content, agent_id, min_similarity=0.95)
        if existing:
            return await self.update_version(existing[0].id, content, confidence)

        embedding = await self.embed(content)
        sanitized = sanitize_context(content)  # 민감정보 제거

        return await self.db.insert(SemanticMemory(
            agent_id=agent_id,
            memory_type=memory_type,
            content=sanitized,
            embedding=embedding,
            source_trace_id=source_trace_id,
            tags=tags,
            confidence=confidence,
        ))
```

# 6. Memory 생명주기 관리

## 6.1 전체 생명주기 흐름

```
[생성]
  Agent 실행 시작 → Session Memory 생성 (Redis)
  실행 완료 → Episodic Memory 저장 (PostgreSQL)
  패턴 학습 → Evol-A 분석 → Semantic Memory 등록 (승인 후)

[사용]
  Agent 실행 전 → Semantic Memory 조회 → 컨텍스트 주입
  실행 중 → Session Memory 상태 갱신
  실패 분석 → Episodic Memory 조회 → 원인 파악

[만료/삭제]
  Session Memory → TTL 자동 만료
  Episodic Memory → 보존 정책 기준 자동 삭제
  Semantic Memory → 신뢰도 하락 시 is_active=FALSE → 승인 후 삭제
```

## 6.2 개인정보 처리 방침

| 항목 | 기준 |
|---|---|
| PII 저장 | 원칙적 금지. 불가피한 경우 암호화 후 저장, 명시적 동의 필요 |
| PII 마스킹 | 저장 전 SENSITIVE_PATTERNS 필터 자동 적용 |
| 열람 권한 | 해당 도메인 Human 책임자 + PM 승인 (audit_log 필수) |
| 삭제 요청 | GDPR/개인정보보호법 기준, P1 ApprovalGate 후 처리 |
| 해외 이전 | 금지 (국내 서버 전용, 공공기관 적용 기준) |
| 보존 기간 | 목적 달성 즉시 최소화, 법정 보존 기간 준수 |

## 6.3 Memory 무결성 보장

```
# Memory 접근 시 항상 아래 규칙 적용
class MemoryIntegrityChecker:


    def validate_access(self, agent_id: str, memory: Memory) -> bool:
        # 1. 해당 Agent의 접근 권한 확인
        if memory.agent_id and memory.agent_id != agent_id:
            raise MemoryAccessError(agent_id, memory.id)


        # 2. is_active 확인
        if not memory.is_active:
            raise MemoryDeprecatedError(memory.id)


        # 3. 보존 기간 만료 확인
        if memory.expires_at and memory.expires_at < datetime.now():
            raise MemoryExpiredError(memory.id)
        return True
```

# 7. Agent 별 Memory 사용 기준

| Agent | Layer 1 | Layer 2 | Layer 3 | Layer 4 |
|---|---|---|---|---|
| Orch | ✓ Workflow 상태 | ✓ 실행 상태 | ✓ 읽기/쓰기 | ✓ 읽기 전용 |
| Monitor-A | ✓ KPI 스냅샷 | ✓ 메트릭 캐시 | ✓ 읽기 전용 | ✕ |
| Back-A | ✓ 코드 컨텍스트 | ✓ 생성 상태 | ✓ 읽기/쓰기 | ✓ 읽기 전용 |
| Front-A | ✓ UI 컨텍스트 | ✓ 컴포넌트 상태 | ✓ 읽기/쓰기 | ✓ 읽기 전용 |
| Test-A | ✓ 시나리오 | ✓ 테스트 상태 | ✓ 읽기/쓰기 | ✓ 읽기 전용 |
| Evol-A | ✓ 분석 컨텍스트 | ✓ 분석 상태 | ✓ 읽기 전용 | ✓ 읽기/쓰기(승인) |
| UIUX-A | ✓ Persona 컨텍스트 | ✓ 설계 상태 | ✓ 읽기/쓰기 | ✓ 읽기 전용 |

# 8. 산출물 체크리스트

| # | 항목 | 완료 |
|---|---|---|
| 1 | 4 계층 Memory 구조 정의 완료 | ☐ |
| 2 | Context Window Manager 설계 완료 | ☐ |
| 3 | Session Memory (Redis) 키 구조 정의 완료 | ☐ |
| 4 | trajectory_logs DB 스키마 확정 완료 | ☐ |
| 5 | semantic_memories DB 스키마 확정 완료 | ☐ |
| 6 | Retrieval Rule 우선순위 정의 완료 | ☐ |
| 7 | 개인정보 처리 방침 정의 완료 | ☐ |
| 8 | Memory 생명주기 및 보존 정책 정의 완료 | ☐ |
| 9 | Agent 별 Memory 접근 권한 매트릭스 완료 | ☐ |
| 10 | Architect + PM 검토 완료 (개인정보 정책 결정) | ☐ |
| 11 | Back-H 검토 완료 (DB 스키마 확정) | ☐ |

> ✅ 11 개 항목 전체 완료 → **Phase 3 Memory Architecture 구현 가능**

# 10. 승인 (Approval)

| 구분 | 내용 | 서명 | 날짜 |
|---|---|---|---|
| 작성자 | | | |
| 검토자<br>(AI Architect) | | | |
| 승인자<br>(Business Owner) | | | |
| 버전 | v1.0 | | |